Credit Card Data Platform

Azure End-to-End ETL Pipeline
Technical Design & Implementation Report

Architecture: **Medallion (Raw → Bronze → Silver → Gold)**
Cloud Platform: **Microsoft Azure**
Orchestration: **Azure Synapse Analytics**
Processing: **PySpark on Synapse Spark Pool (Spark 3.5)**
Storage: **Azure Data Lake Storage Gen2 + Delta Lake**
Serving: **Synapse Dedicated SQL Pool**

Document Version 1.0 | May 2026

1. Executive Summary

This document provides a comprehensive technical design and implementation record for the Credit Card Data Platform — an end-to-end cloud data engineering solution built on the Microsoft Azure ecosystem. The platform ingests raw credit card customer and transaction data, processes it through a structured Medallion Architecture (Raw → Bronze → Silver → Gold), and delivers analytics-ready aggregations to a Synapse Dedicated SQL Pool for business intelligence consumption.

The solution was designed to address common challenges in financial data engineering: handling large volumes of transactional data reliably, enforcing data quality at each layer, managing slowly changing dimensions, and making curated datasets available to downstream consumers with minimal latency.

Key Outcomes
Fully automated ETL pipeline · Medallion Architecture across 4 layers · Schema enforcement at every boundary · SCD Type 2 implementation · Three business-level aggregated Gold tables · Secure credential management via Azure Key Vault · Analytics-ready data served directly to Synapse SQL Pool

2. Business Objective

Financial institutions require a reliable, scalable pipeline to transform raw credit card operational data into curated datasets that support strategic decision-making. The Credit Card Data Platform was built to fulfil the following business needs:

- Customer Spend Analytics — understand how customers use their credit cards across months, card types, and spend categories
- Merchant Intelligence — identify top-performing merchants and merchant categories by revenue, volume, and average transaction value
- Geographic Insights — analyse spending patterns by merchant country to inform international product and risk strategy
- Data Reliability — ensure downstream consumers receive clean, deduplicated, and consistently structured data with full audit traceability
- Operational Efficiency — automate the full ingestion-to-serving pipeline to eliminate manual data handling

3. Solution Architecture

3.1 Medallion Architecture Overview

The platform implements the Medallion Architecture pattern — a widely adopted data lakehouse design that organises data into progressively refined layers. Each layer has a distinct purpose and applies increasing levels of transformation and business logic.

RAW	BRONZE	SILVER	GOLD	SQL POOL
Landing Zone	**Ingested Data**	**Cleaned Data**	**Aggregations**	**Analytics Serving**
CSV files land in ADLS Gen2 raw/credit-card/ container. Immutable source of truth.	Synapse Copy Activity moves CSVs to bronze layer with schema validation.	PySpark transforms: dedup, null handling, type casting, derived columns, SCD2.	Business aggregations: customer spend, merchant summary, country summary.	Gold tables written via JDBC to Synapse Dedicated SQL Pool for BI consumption.

3.2 Storage Layout (ADLS Gen2)

All data is stored in Azure Data Lake Storage Gen2 on the storage account retailsalesgb.dfs.core.windows.net. The following container and folder structure organises data by layer:

```
retailsalesgb.dfs.core.windows.net
│
├── raw/                          ← Raw landing container
│   └── credit-card/              ← Source CSV files (customers, transactions)
│
└── credit-card/                  ← Processed data container
    ├── bronze/                   ← Ingested CSVs (Copy Activity output)
    ├── silver/                   ← Delta Lake (customers, transactions)
    └── gold/                     ← Delta Lake (aggregations + curated join)
```

3.3 Azure Services Used

Service	Role
Azure Data Lake Storage Gen2	Scalable hierarchical storage for all raw, bronze, silver, and gold data
Azure Synapse Analytics	Unified analytics platform providing pipeline orchestration and SQL serving
Azure Synapse Pipelines	ETL orchestration — Copy Activity and Notebook Activity sequencing
Azure Synapse Spark Pool	Distributed PySpark processing engine (Spark 3.5, Memory Optimised)
Azure Synapse Dedicated SQL Pool	Analytics serving layer — stores Gold aggregation tables for BI queries
Azure Key Vault	Secure secret management — SQL username and password retrieved at runtime
Delta Lake	ACID-compliant open table format used for Silver and Gold layers

AutoResolve Integration Runtime	Managed integration runtime for Copy Activity execution

4. Data Pipeline: RawData2Bronze

The pipeline RawData2Bronze is the sole orchestration unit in the platform. It is defined in Azure Synapse Pipelines and executes two sequential activities that together cover the full data flow from raw ingestion to Gold serving.

4.1 Activity 1 — Copy Activity (Raw → Bronze)

The Copy Activity transfers all CSV files from the Raw layer to the Bronze layer. It uses a wildcard pattern (*) with recursive traversal to pick up all files in the source folder, making the pipeline resilient to multi-file drops and future additions without configuration changes.

Property	Value
Source	SourceDataset_1b0 → raw/credit-card/ (ADLS Gen2)
Destination	DestinationDataset_1b0 → credit-card/bronze/ (ADLS Gen2)
File Pattern	Wildcard: * (recursive traversal enabled)
Format	Delimited Text (CSV), comma separator, header row, quote-escaped
Output Extension	.csv
Timeout	12 hours
Retry Policy	0 retries, 30-second retry interval
Linked Services	ls_raw2bronze (source), ls_CC_bronze (destination)
Int. Runtime	AutoResolveIntegrationRuntime (Managed, AutoResolve location)

4.2 Activity 2 — Synapse Notebook Activity (Transformations)

On successful completion of the Copy Activity, the pipeline triggers the Transformations PySpark notebook on the Synapse Spark Pool. This notebook performs all Bronze → Silver → Gold transformations and writes results to both Delta Lake and the Synapse SQL Pool.

Property	Value
Notebook	Transformations
Spark Pool	SparkSynpPool (Spark 3.5, Small nodes, Memory Optimised)
Auto-Scale	Min 3 / Max 3 nodes
Auto-Pause	After 15 minutes of inactivity
Dependency	Executes only after Copy Activity succeeds

Secret Source	Azure Key Vault — CC-KeyVault-SQL (SQLPasswordCC, UserNameSQL)
Timeout	12 hours

5. Data Schema

5.1 Customers Dimension (dim_customers.csv)

The customers dataset contains demographic, card, and financial profile information per credit card holder. It serves as the dimension table joined with transaction fact data in the Gold layer.

Field	Type	Description
customer_id	String	Unique customer identifier (primary key)
card_id	String	Unique card identifier
first_name	String	Customer first name (normalised to title case)
last_name	String	Customer last name (normalised to title case)
gender	String	Customer gender
date_of_birth	Date	Date of birth — used to derive customer_age
email	String	Email address (normalised to lowercase)
phone	String	Contact phone number
address_line1	String	Primary address
city	String	City of residence
postcode	String	Postcode
country	String	Country of residence
occupation	String	Customer occupation
annual_income_gbp	String	Annual income in GBP
credit_score	String	Credit score — filtered to values > 0
card_type	String	Card product type (e.g. Platinum, Gold)
card_number_masked	String	Masked card number (last 4 digits visible)
card_issue_date	String	Card issue date
card_expiry_month_year	String	Card expiry month and year
credit_limit_gbp	String	Approved credit limit in GBP
card_status	String	Active / Suspended / Cancelled — used as partition key in Silver
rewards_points	String	Accumulated rewards points balance
customer_since	Date	Date customer joined — cast to Date in Silver

5.2 Transactions Fact (fact_transactions.csv)

The transactions dataset captures individual credit card transaction events. Schema enforcement using a PySpark StructType is applied at the Bronze read stage to ensure type consistency.

Field	Type	Description
transaction_id	String	Unique transaction identifier (primary key, deduplicated)
card_id	String	Card used for the transaction
customer_id	String	Customer who made the transaction (join key)
transaction_date	Date	Date of transaction (cast from String, yyyy-MM-dd)
transaction_time	String	Time of transaction — combined with date to form timestamp
transaction_year	Integer	Year extracted — used as Silver partition key
transaction_month	Integer	Month extracted — used as Silver partition key
transaction_day	Integer	Day of month
transaction_type	String	Purchase / Refund / Cash Advance / etc.
channel	String	Online / In-store / Contactless / etc.
merchant_name	String	Name of the merchant
merchant_category_code	Integer	MCC code (ISO 18245)
merchant_category_name	String	Human-readable merchant category
merchant_city	String	City of merchant
merchant_country	String	Country of merchant — Gold aggregation dimension
amount_gbp	Double	Transaction amount in GBP — filtered to > 0
currency	String	Transaction currency
is_foreign_transaction	Boolean	Flag for cross-currency transactions
fx_rate	Double	FX rate applied (if foreign transaction)
transaction_status	String	Approved / Declined — drives is_declined flag
decline_reason	String	Reason for decline; null-filled to N/A if not declined
is_recurring	Boolean	Recurring payment flag
installment_plan	Boolean	Instalment plan flag
cashback_gbp	Double	Cashback earned in GBP
rewards_earned	Integer	Rewards points earned on transaction
authorization_code	String	Acquirer authorisation code

6. Transformation Logic (Bronze → Silver)

All transformations are implemented in PySpark within the Transformations notebook, executed on the Synapse Spark Pool. Transformations are applied sequentially, progressing from data quality enforcement to enrichment and finally aggregation.

6.1 Transaction Cleaning

- Deduplication — dropDuplicates(["transaction_id"]) removes exact duplicate transaction records
- Amount filter — filter(col("amount_gbp") > 0) excludes zero-value and erroneous records
- Date casting — transaction_date cast from String to Date using format yyyy-MM-dd
- Timestamp construction — transaction_timestamp derived by concatenating transaction_date and transaction_time via concat_ws
- Null imputation — decline_reason filled with N/A for non-declined transactions using fillna

6.2 Customer Cleaning

- Deduplication — dropDuplicates(["customer_id"]) ensures one record per customer
- Name normalisation — first_name and last_name converted to title case using initcap(trim(...))
- Email standardisation — email converted to lowercase using lower(trim(...))
- Date casting — date_of_birth and customer_since cast from String to Date
- Credit score filter — filter(col("credit_score") > 0) removes invalid customer records

6.3 Derived Columns

Column	Logic
transaction_category	Categorises each transaction as High Value (≥ £1,000), Medium Value (≥ £500), or Low Value (< £500) using PySpark when/otherwise logic
is_declined	Binary flag (1/0) derived from transaction_status == 'Declined' — used in the customer_monthly_spend declined_transactions aggregation
customer_age	Integer age in years, computed as floor(datediff(current_date(), date_of_birth) / 365)
transaction_timestamp	Full datetime combining transaction_date and transaction_time using concat_ws and to_timestamp

6.4 SCD Type 2 — Customer Dimension

The customer dimension is designed with SCD Type 2 metadata to support historical tracking of customer attribute changes. The following columns are added before writing to the Silver layer:

Column	Value / Purpose
start_date	Set to current_date() — the effective start date of the record version
end_date	Set to null — indicates this is the currently active record

is_current	Set to true — flag identifying the active record version

6.5 Silver Layer Write

Silver data is written in Delta Lake format with overwriteSchema enabled to support iterative development. Partitioning is applied to optimise query performance for downstream reads:

- Customers — partitioned by card_status (Active, Suspended, Cancelled)
- Transactions — partitioned by transaction_year and transaction_month

7. Gold Layer — Aggregations & Serving

7.1 Enriched Dataset

Transactions and Customers are joined on customer_id using a left join, producing a wide enriched dataset (df_enriched) that forms the base for all Gold aggregations. The card_id column is dropped post-join to avoid ambiguity.

```
df_enriched = silver_trans.join(silver_cust, on=["customer_id"], how="left")
df_enriched = df_enriched.drop("card_id")
# Written to: credit-card/gold/curated_cc_data (Delta Lake)
```

7.2 Gold Aggregations

customer_monthly_spend

Aggregates transaction activity per customer per calendar month, enabling customer spend trend analysis and decline rate monitoring.

Field	Type	Description
customer_id	String	Customer identifier (group key)
transaction_year	Integer	Year of transactions (group key)
transaction_month	Integer	Month of transactions (group key)
total_spend_gbp	Float	Sum of amount_gbp, rounded to 2 decimal places
total_transactions	Integer	Count of transaction records
avg_transaction_amount	Float	Average of amount_gbp, rounded to 2 decimal places
declined_transactions	Integer	Sum of is_declined flag

merchant_summary

Aggregates revenue and volume metrics per merchant and merchant category, supporting merchant performance reporting.

Field	Type	Description
merchant_name	String	Merchant name (group key)
merchant_category_name	String	Merchant category (group key)
total_revenue	Float	Sum of amount_gbp, rounded to 2 decimal places
transaction_count	Integer	Count of transactions
avg_ticket_size	Float	Average of amount_gbp, rounded to 2 decimal places

country_summary

Aggregates total spend and unique customer reach per merchant country, supporting geographic and international product analysis.

Field	Type	Description
merchant_country	String	Merchant country (group key)
country_spend	Float	Sum of amount_gbp, rounded to 2 decimal places
unique_customers	Integer	Count of distinct customer_id values

7.3 SQL Pool Table Definitions

Gold aggregation tables are pre-created in the gold schema of the Synapse Dedicated SQL Pool with ROUND_ROBIN distribution and HEAP storage, optimising for flexible analytics workloads without a predefined distribution key requirement.

```
CREATE SCHEMA gold;

CREATE TABLE gold.country_summary (
    merchant_country  NVARCHAR(100),
    country_spend     FLOAT,
    unique_customers  INT
) WITH (DISTRIBUTION = ROUND_ROBIN, HEAP);

CREATE TABLE gold.merchant_summary (
    merchant_name           VARCHAR(255),
    merchant_category_name  VARCHAR(255),
    total_revenue           FLOAT,
    transaction_count       INT,
    avg_ticket_size         FLOAT
) WITH (DISTRIBUTION = ROUND_ROBIN, HEAP);

CREATE TABLE gold.customer_monthly_spend (
    customer_id             VARCHAR(50),
    transaction_year        INT,
    transaction_month       INT,
    total_spend_gbp         FLOAT,
    total_transactions      INT,
```

```
    avg_transaction_amount  FLOAT,
    declined_transactions   INT
) WITH (DISTRIBUTION = ROUND_ROBIN, HEAP);
```

7.4 JDBC Write to SQL Pool

Gold aggregations are written from the Spark Pool to the Synapse Dedicated SQL Pool via JDBC in append mode. Credentials are retrieved securely at runtime from Azure Key Vault using mssparkutils.credentials.getSecret, ensuring no secrets are stored in notebook code.

```
jdbc_url = "jdbc:sqlserver://az-synapse-project.sql.azuresynapse.net:1433;
            database=SynapseSQLPool"

df.write.format("jdbc")
    .option("url",      jdbc_url)
    .option("dbtable",  "gold.<table_name>")
    .option("user",     username)   # from Key Vault
    .option("password", password)   # from Key Vault
    .option("driver",   "com.microsoft.sqlserver.jdbc.SQLServerDriver")
    .mode("append")
    .save()
```

8. Data Quality Framework

Data quality is enforced at every layer boundary through a combination of schema enforcement, filtering, cleansing, and standardisation rules.

Check	Implementation
Deduplication	dropDuplicates on transaction_id and customer_id eliminates exact duplicate records at the Bronze→Silver boundary
Amount Validity	Transactions with amount_gbp ≤ 0 are excluded to prevent erroneous or test records from reaching the Silver layer
Credit Score Validity	Customers with credit_score ≤ 0 are excluded as invalid records
Schema Enforcement	Transactions are read with an explicit PySpark StructType, enforcing column names and data types at the Bronze read stage
Schema Evolution	overwriteSchema = true on Delta Lake writes supports schema updates during iterative development without breaking the pipeline
Null Imputation	decline_reason is null-filled to N/A for non-declined transactions to avoid nulls in downstream aggregations
Type Casting	date_of_birth, customer_since, and transaction_date are explicitly cast to Date; transaction_timestamp to Timestamp
String Normalisation	Names standardised to title case; email to lowercase; leading/trailing whitespace removed via trim
Partitioning	Silver tables partitioned by relevant keys (card_status, transaction_year/month) to reduce query scan footprint

9. Security & Credential Management

The platform follows Azure security best practices for credential handling and data access control.

- All ADLS Gen2 linked service credentials are stored as encrypted credentials within Azure Synapse, never in plain text in pipeline definitions
- SQL Pool username and password are stored in Azure Key Vault (CC-KeyVault-SQL) and retrieved at notebook runtime using mssparkutils.credentials.getSecret — no secrets appear in source code
- Linked services (ls_raw2bronze, ls_CC_bronze) are scoped to specific containers, applying least-privilege access to storage
- The AutoResolve Integration Runtime is a Microsoft-managed compute with no customer-managed infrastructure exposure

10. Infrastructure Configuration

10.1 Synapse Spark Pool

Setting	Value
Pool Name	SparkSynpPool
Spark Version	3.5
Node Size	Small
Node Family	Memory Optimised
Auto-Scale	Enabled (Min 3 / Max 3 nodes)
Auto-Pause	Enabled — pauses after 15 minutes of inactivity
Region	Japan East
Isolation	Compute isolation disabled
Session Packages	Session-level packages disabled

10.2 Integration Runtime

Setting	Value
Name	AutoResolveIntegrationRuntime
Type	Managed (Microsoft-managed compute)

Location	AutoResolve (Azure selects optimal region)
Compute Type	General Purpose
Core Count	8 cores
Time-to-Live	0 minutes (spins up fresh per execution)

11. Key Engineering Outcomes

Outcome	Detail
End-to-End Automation	Single pipeline trigger ingests, transforms, and serves data across all four Medallion layers with no manual intervention
Scalable Architecture	Medallion pattern provides clear separation of concerns and makes each layer independently scalable and testable
Schema Enforcement	Explicit StructType definitions prevent malformed data from propagating beyond the Bronze read boundary
Delta Lake Reliability	ACID-compliant Delta Lake format provides transactional writes, schema enforcement, and supports future time-travel queries
SCD Type 2 Readiness	Customer dimension includes start_date, end_date, and is_current scaffolding for full historical change tracking
Secure Credential Handling	Azure Key Vault integration ensures zero credential exposure in code, configuration files, or logs
Analytics-Ready Gold Layer	Three aggregated Gold tables served directly to Synapse SQL Pool, queryable by BI tools without additional transformation
Cost Efficiency	Spark Pool auto-pause (15 min) and auto-scale minimise compute cost during idle periods
Modular Design	Notebook, pipeline, and linked service definitions are independently version-controllable and redeployable